

ASSIGNMENT 3

Software Development and Integration

REPORT

climate-analyzer: a scientific tool for analysing and forecasting the weather of Astana

Student: Aibyn Talgatov

Repository: github.com/a169n/applied_software_3

Astana, 2026

CONTENTS

Introduction	2
1 Stages of the software development process	2
2 Criteria for choosing technologies.....	4
3 Version control systems: Git and GitHub compared with alternatives	5
4 CI/CD concepts and their implementation	6
5 Features of scientific software development	7
5.1 Results for Astana on real data	8
6 Links to the repository and CI/CD configuration.....	10
Conclusion.....	10
References	11

Introduction

The purpose of this assignment is to take a small piece of scientific software through its complete cycle, from design to automated testing and delivery, and to justify the technologies chosen. The practical result is **climate-analyzer**, an open-source Python package published on GitHub [1]. It loads daily weather records, removes defective values, estimates long-term trends, anomalies and extremes, and forecasts the coming months. The case study is Astana, whose sharply continental climate (about $-14\text{ }^{\circ}\text{C}$ in January and $+21\text{ }^{\circ}\text{C}$ in July) makes both the seasonal cycle and the data-quality problems easy to see.

The tool works as a command line program (`climate-analyzer fetch and analyze`), an interactive Streamlit web interface (`app.py`) and a Python library. The repository also holds a README, an MIT licence, issue and pull request templates, a changelog, 84 automated tests and three GitHub Actions workflows. Unless stated otherwise, the numerical results below were obtained by running the tool on real data for Astana.

1 Stages of the software development process

Software engineering describes the creation of a program as a life cycle of stages: requirements analysis, design, implementation, testing, deployment and maintenance [2]. Life-cycle models differ in how the stages are ordered. In the classical *waterfall* model each stage is finished before the next begins, which is easy to plan but makes late changes expensive. *Iterative* and *agile*

models repeat the stages in short cycles, so a working version exists from the start and feedback arrives early. Scientific software is usually developed iteratively, because the requirements are refined as the researcher learns more about the data [6]. The repository history is organised the same way: a series of small commits, each adding one layer, from the project skeleton and the sample data through the analysis modules, the forecast model and the interfaces to the tests, the CI/CD workflows and the documentation.

Requirements. The functional requirements follow from the task: read a CSV file of daily observations, clean it, compute trends, anomalies and extremes, forecast future months and write a report. The non-functional ones mattered as much: reproducible results, no need for a display or internet access on a CI server, and installation with one command on Linux, Windows and macOS.

Design and implementation. The package consists of small modules with one responsibility each — io, sources, cleaning, analysis, forecast, plotting — joined by a pipeline module. The command line and web interfaces are thin layers over that pipeline, so both always give identical numbers. Network access is isolated in one function that receives its opener as a parameter and can therefore be tested offline. The code is typed Python in the standard src/ layout; Ruff enforces style and mypy checks types.

Testing. The pytest suite has 84 tests covering 99 % of the statements: unit tests for every module, integration tests of the pipeline and the command line program, and tests of the web interface through Streamlit's AppTest. The central idea is testing against a known answer: the bundled synthetic dataset is generated with a warming of +0.35 °C per decade, and the tests check that the analysis recovers it.

Deployment and maintenance. A version tag starts a workflow that tests the code, builds the package and publishes a GitHub Release. Maintenance relies on the issue tracker, the changelog, semantic versioning and Dependabot. Table 1 maps each stage to the repository.

Table 1 – Stages of the development process and their artefacts in the project

Stage	What was done	Artefacts in the repository
Requirements	Functional and non-functional requirements, known limitations	README.md, issue templates
Design	Modular architecture, shared pipeline, injectable network layer	src/climate_analyzer/, docs/TECHNOLOGY.md
Implementation	Typed package, CLI and web UI	pyproject.toml, cli.py, app.py
Testing	84 tests, 99 % coverage, lint, type checks	tests/, .github/workflows/ci.yml
Deployment	Automated build and release on a version tag	.github/workflows/release.yml
Maintenance	Scheduled forecast, dependency updates, changelog	astana-forecast.yml, dependabot.yml, CHANGELOG.md

2 Criteria for choosing technologies

Every dependency is a long-term obligation, so the stack was selected by explicit criteria rather than by habit. They are documented in the repository together with the rejected alternatives (docs/TECHNOLOGY.md):

- **fitness for scientific work** — correct, well-tested numerical methods;
- **ecosystem and community** — maintained, documented libraries that researchers already know;
- **reproducibility** — the same input gives the same output on any machine, including a CI server;
- **low barrier to entry** — free and open source, cross-platform, easy to install;
- **testability and automation** — good support for automated tests, static analysis and CI/CD.

Python meets all five. R is equally strong in statistics but weaker as a general-purpose language for command line tools, packaging and web interfaces; MATLAB needs a commercial licence; Julia is fast but its ecosystem and CI tooling are less mature. Table 2 lists the main components and the reason for each choice.

Table 2 – Selected technologies and the reasons for choosing them

Technology	Role	Reason; rejected alternatives
pandas, NumPy	Time series, numerics	Calendar-aware resampling, native missing values, least squares.
SciPy	Statistical tests	Reviewed implementations of regression, Kendall's tau and the Theil–Sen slope; hand-written versions would risk errors.
Matplotlib	Figures	Publication-quality images rendered without a display.
Streamlit	Web interface	An interactive application in one Python script; Dash or Flask need far more code.
Plotly	Web charts	Hover, zoom and legend toggling without extra code.
Open-Meteo	Data source	Free, no API key, any coordinates, no gaps; station archives (GHCN, Meteostat) have gaps and heavier formats.
pytest, Ruff, mypy	Quality control	Concise tests; one fast tool instead of flake8, isort and black; static detection of type errors.
GitHub Actions	CI/CD	Built into GitHub, free for public repositories, three operating systems; Jenkins needs its own server.

Decisions *not* to use a technology followed the same criteria. The statsmodels library offers seasonal decomposition and ARIMA, but it is a heavy dependency, while the decomposition and the

forecast model take fewer than 200 transparent, fully tested lines here. Deep learning was rejected for a scientific reason: 32 years of monthly data give only 384 points, far too few for a neural network, whereas an interpretable statistical model can be validated honestly against a simple baseline.

3 Version control systems: Git and GitHub compared with alternatives

A version control system (VCS) records every change of the source code with its author, date and explanation. It allows a return to any earlier state, parallel work on several ideas and the merging of contributions. For scientific software it also documents exactly which version of the code produced a published result.

There are two families of systems. In *centralised* systems such as Subversion (SVN) the history is stored on one server and developers hold only a working copy, so most operations need a network connection and the server is a single point of failure. In *distributed* systems such as Git and Mercurial every clone contains the complete history; commits, branches and comparisons are local and fast, and any clone can restore the project [3]. Table 3 compares the three best-known open-source systems.

Table 3 – Comparison of version control systems

Criterion	Git	Subversion (SVN)	Mercurial
Architecture	Distributed	Centralised	Distributed
Work offline	Full	A server is needed to commit	Full
Branching and merging	Very cheap; basis of the pull request workflow	Branches are directory copies; merging is harder	Cheap; named branches and bookmarks
Integrity of history	Objects addressed by a cryptographic hash	Sequential revision numbers	Hash-based
Large binary files	Weak without Git LFS	Good, with file locking	Weak without extensions
Learning curve	Steep	Gentle	Gentle
Hosting and ecosystem	GitHub, GitLab, Bitbucket; de facto standard	Mostly corporate servers	Few hosts; Bitbucket dropped it in 2020

Git was chosen for three reasons. Cheap local branches make it natural to isolate every change and verify it before it reaches the main branch. Content hashing guarantees that a commit identifier refers to exactly one state of the code, which matters for reproducibility. And Git is the standard of both industry and open science, so most users and reviewers already know it. Its

weaknesses are not significant here: the repository holds only source code and a few small data and image files.

Git itself only manages the history of a repository; a hosting platform adds the collaboration layer. In this project **GitHub** provides the public repository, the issue tracker with templates, pull requests, releases with downloadable packages, Dependabot and the CI/CD service. Of the alternatives, GitLab offers a comparable built-in CI/CD system and can be self-hosted, which suits closed institutional projects, and Bitbucket integrates with Jira and the other Atlassian tools and is aimed at company teams. GitHub was preferred because it hosts the largest open-source and scientific community, its Actions service is free for public repositories, and a repository can be archived with a citable DOI through Zenodo.

4 CI/CD concepts and their implementation

Continuous integration (CI) is the practice of merging changes into the main branch frequently and verifying each one automatically by building the project and running its tests [4], so that errors are found within minutes. **Continuous delivery (CD)** extends the idea: after successful verification the software is automatically prepared for release, so publishing a version becomes a routine, repeatable action; in *continuous deployment* even the final step is automatic [5]. The sequence of automated steps is a *pipeline*, and its outputs are stored as *artefacts*.

In GitHub Actions a pipeline is a YAML file in `.github/workflows` [12]. A *workflow* is started by an *event* (a push, a pull request, a schedule, a tag or a manual start) and consists of *jobs* that run on clean virtual machines called *runners*. Because the configuration lives in the repository, the pipeline is versioned and reviewed together with the code. The project has three workflows (Table 4).

Table 4 – GitHub Actions workflows of the project

Workflow	Trigger	What it does
ci.yml	Push to main, pull request, manual start	Ruff and mypy checks; tests on Ubuntu (Python 3.10–3.13), Windows and macOS; end-to-end analysis of the sample; package build and install check.
astana-forecast.yml	Monthly schedule, manual start	Downloads real Astana data, analyses temperature and precipitation, publishes the reports as a job summary and an artefact.
release.yml	Tag v*	Checks that the tag matches the package version, runs the tests, builds the package and creates a GitHub Release.

The CI workflow uses several standard techniques. A **build matrix** repeats the tests in six environments — four Python versions on Ubuntu plus Windows and macOS — and so catches

errors that appear on one platform only. **Job dependencies** form the stages: the end-to-end analysis and the package build start only after the quality checks pass. A **quality gate** fails the run if coverage drops below 90 %. **Concurrency control** cancels an outdated run when a newer commit arrives, and **least-privilege permissions** give the CI token read-only access. Figure 1 shows run #11 on the main branch (commit dcbbbb30): all nine jobs finished successfully in 2 minutes 57 seconds.

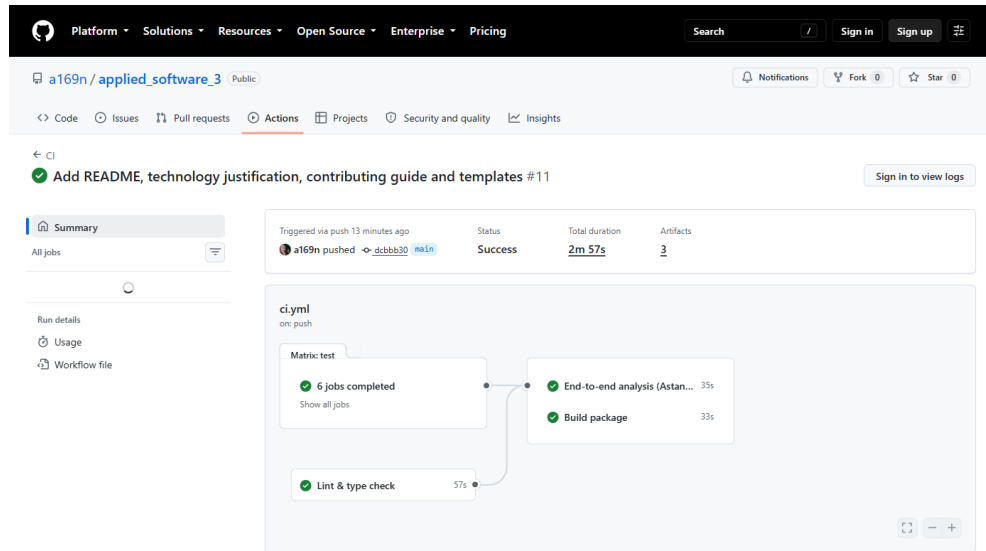


Figure 1 – Successful run #11 of the CI workflow on the main branch

The second workflow uses the CI infrastructure for a scientific task rather than for checking code: every month, or on demand through `workflow_dispatch`, it repeats the whole study on fresh data on an independent machine.

The third workflow implements continuous delivery. When a version tag is pushed, it checks that the tag matches the package version, repeats the tests, builds the wheel and the source archive and publishes them as a GitHub Release. The workflow therefore cannot publish code that fails its tests; version 0.1.0 was released this way.

5 Features of scientific software development

Scientific software differs from ordinary applications in what counts as a defect. A business application that crashes is obviously broken; a scientific program that silently returns a plausible but wrong number is far more dangerous, because the error passes into publications. Research software engineering therefore stresses a specific set of practices [6, 7], each of which was applied here.

Correctness against a known answer. For real climate data the true trend is unknown, so a result cannot be checked directly. The project solves this *oracle problem* with a synthetic dataset that imitates Astana's climate, has a known warming and contains deliberate defects (gaps, sensor spikes, -999 codes). The tests require the tool to remove the defects and recover the known trend.

Reproducibility. The sample is generated with a fixed random seed, figures are rendered without a display, tests never use the network, and the test suite turns every FutureWarning — the category pandas and NumPy use to announce coming changes of behaviour — into an error, so that a library update cannot alter the results unnoticed.

Domain-aware data quality. Generic rules fail on climate data: a global outlier threshold would mark every Astana winter day as an anomaly, so outliers are judged against the same calendar month. Gaps of up to three days are interpolated, longer ones are left empty so that weather is not invented, and a month or year with less than 80 % of its days is excluded.

Honest treatment of uncertainty. The trend is estimated twice, by ordinary least squares and by the non-parametric Mann–Kendall test with Sen's slope [8, 9], the standard robust method in climatology. The forecast has a 95 % prediction interval and is scored on data the model has not seen, against the naive climatology forecast (“an average January”).

5.1 Results for Astana on real data

The tool was run on 11 688 daily values for 1 January 1994 – 31 December 2025, downloaded without gaps from the Open-Meteo archive [10]. By default the archive combines the ERA5 and ERA5-Land reanalyses [11] (used here up to 2016) with the operational ECMWF IFS model (from 2017), and it describes a model grid cell near 51.1° N, 71.4° E rather than one weather station. The whole-record means — −14.4 °C in January, +20.6 °C in July — agree with the published 1991–2020 station normals of −14.5 and +20.6 °C [13]. Table 5 gives the main results.

Table 5 – Analysis of Astana air temperature, 1994–2025

Indicator	Value
Mean daily temperature	4.10 °C
Coldest / warmest year	1996 (1.29 °C) / 2023 (5.72 °C)
Coldest / warmest daily mean	−39.6 °C (2012-12-15) / +31.6 °C (2008-07-30)
Linear trend (OLS)	+0.35 ± 0.16 °C per decade (± standard error), p = 0.040
Sen's slope; Mann–Kendall test	+0.30 °C per decade; $\tau = 0.21$, p = 0.089
Cold days per year, first / second half of the record	36.7 / 30.7
Forecast model: variance explained; residual std	$R^2 = 0.96$; 2.63 °C
Backtest, 36 months: MAE of model / climatology	1.80 °C / 1.91 °C (skill +6.0 %)
Forecast for January / July 2026 (95 % interval)	−13.9 °C (−19.0 ... −8.8) / +21.7 °C (+16.5 ... +26.8)

The annual mean temperature rose by +0.35 °C per decade (Figure 2). This is of the same order as the 0.33 °C per decade that Kazhydromet reports for the Akmolra region over the longer

period 1976–2023 [14] (the equal value built into the synthetic sample is a coincidence). The result shows why two tests are reported. Least squares gives $p = 0.040$, formally below 0.05, but the more cautious Mann–Kendall test gives $p = 0.089$: the least-squares slope is pulled up by the very cold 1996 at the start of the record, and without that single year it falls to $+0.23$ °C per decade ($p = 0.14$). The tool therefore reports the trend as *not statistically significant* at the 5 % level; a program that printed only the first p-value would have overstated the evidence. The number of cold days per year (days below the 10th percentile of the same calendar month in the baseline) fell from 37 in 1994–2009 to 31 in 2010–2025, while warm days (above the 90th percentile) hardly changed (36 and 38), so the warming appears mainly as a loss of cold extremes.

The forecast model — a linear trend, two annual harmonics and first-order autoregressive residuals — explains 96 % of the variance of monthly temperature. In the backtest it was fitted on data up to December 2022 and compared with the next 36 months: its mean absolute error was 1.80 °C against 1.91 °C for climatology, and 97 % of the observations fell inside the 95 % interval (Figure 3). The gain of 6 % is modest, and the README states the reason: a statistical model cannot predict individual weather events months ahead. An out-of-sample check is already possible: against Open-Meteo values for January–September 2026, retrieved separately on 6 October 2026, all nine months fell inside the forecast interval, with a mean absolute error of 2.05 °C against 2.28 °C for climatology. For precipitation the same model explains only 25 % of the variance. The apparent increase of annual totals is not significant (Mann–Kendall $p = 0.19$) and is inflated by the 2017 change of the underlying model, so it is not interpreted as a climate signal. For temperature the change matters little: ERA5-Land alone gives $+0.34$ °C per decade (Mann–Kendall $p = 0.10$), so the cautious verdict does not depend on it.

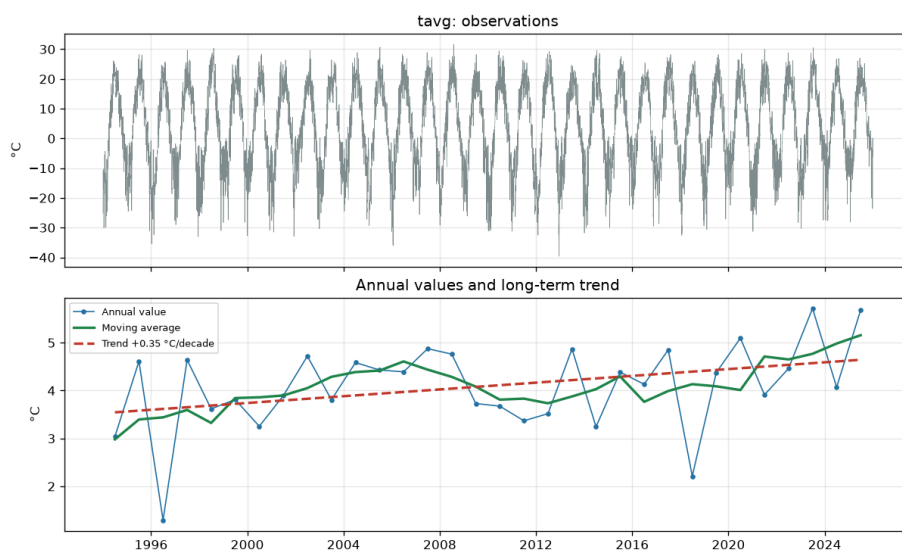


Figure 2 – Daily temperature of Astana and the trend of annual means (real data, 1994–2025)

Real data also exposed a weakness that the synthetic tests could not. The only value that the outlier filter removed, -33.4 °C on 28 November 1998, was a genuine cold wave confirmed by

station records, not an error. The effect on the trend is negligible, but a rule designed for sensor faults should not be applied blindly to reanalysis data.

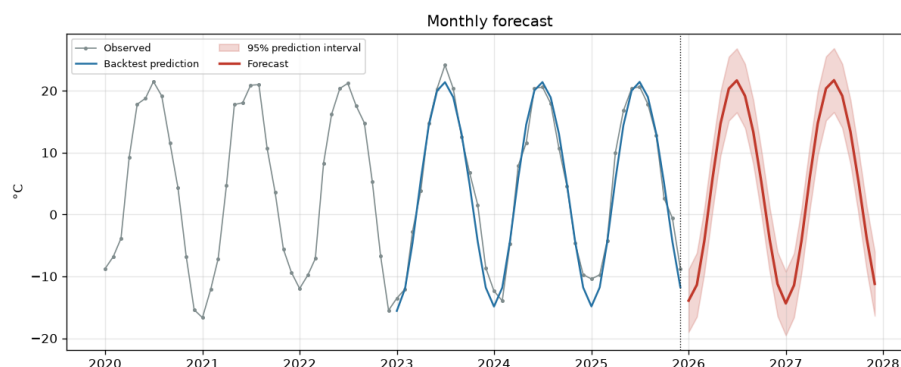


Figure 3 – Backtest and 24-month forecast of monthly mean temperature with the 95 % prediction interval

6 Links to the repository and CI/CD configuration

All materials of the project are public (Table 6). In the PDF every entry is a link; file paths are relative to the main branch of the repository.

Table 6 – Project links

Item	Link
Repository	github.com/a169n/applied_software_3
Justification of technologies	docs/TECHNOLOGY.md
CI workflow	.github/workflows/ci.yml
Forecast workflow	.github/workflows/astana-forecast.yml
Release workflow	.github/workflows/release.yml
CI run #11 (Figure 1)	actions/runs/3742325013
Release v0.1.0	releases/tag/v0.1.0
Issues	github.com/a169n/applied_software_3/issues

Conclusion

A small scientific tool was taken through every stage of the software development process. The result is a documented, licensed open-source package with 84 automated tests and a CI/CD pipeline that checks every change on three operating systems, repeats the analysis on fresh data and publishes releases. Applied to real data for Astana for 1994–2025, the tool estimated a warming of +0.35 °C per decade and reported, by the more cautious Mann–Kendall test, that this trend is not statistically significant at the 5 % level. It also produced a forecast whose skill and uncertainty were measured rather than assumed.

The main challenges encountered during the work were the following.

- **No access to the data source during development.** The environment in which the code was first written could not reach the Open-Meteo API. The downloader was therefore designed with an injectable opener and tested on prepared responses, and a synthetic dataset was created. The real download, made later, matched the API response exactly.

- **Real data behaves differently from test data.** Besides the outlier case above, the first real run showed that the archive changes its underlying model in 2017; the README, written earlier, described the source simply as ERA5 and had to be corrected. Tests on synthetic data could reveal neither problem.

- **Machine-specific failures.** On the Windows development machine the download failed with a TLS error although the server certificate was valid: the system store held an expired intermediate certificate that Python preferred to the valid chain. On that machine the download was run with the truststore package, which delegates verification to the operating system, instead of switching the check off; the published code was left unchanged. A security policy also blocked the launcher programs that pip generates, so the tools were started as `python -m`.

- **Interpreting statistics honestly.** Least squares and the Mann–Kendall test disagreed about the significance of the trend, so the rule for the verdict had to be fixed in advance rather than chosen after seeing the result.

- **The pipeline needs maintenance too.** Within two minutes of the first push Dependabot opened eight update pull requests. They were closed without merging for now, so the upgrade of actions/upload-artifact, which would remove GitHub's warning about a deprecated Node.js version, remains the first maintenance task.

The main lesson is that version control, automated testing and continuous integration and delivery are not an addition to scientific work but a condition of its reliability: a result recomputed automatically on an independent machine, with tests that know the right answer, can be trusted and reproduced by others.

References

1. Talgatov A. climate-analyzer: source code repository. GitHub, 2026. URL: https://github.com/a169n/applied_software_3
2. Sommerville I. Software Engineering. 10th ed. Pearson, 2016. 816 p.
3. Chacon S., Straub B. Pro Git. 2nd ed. Apress, 2014. URL: <https://git-scm.com/book>
4. Fowler M. Continuous Integration. 2000 (rewritten 2006, revised 2024). URL: <https://martinfowler.com/articles/continuousIntegration.html>

5. Humble J., Farley D. Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation. Addison-Wesley, 2010. 512 p.
6. Wilson G. et al. Best Practices for Scientific Computing // PLoS Biology. 2014. Vol. 12, No. 1. e1001745.
7. Wilson G. et al. Good Enough Practices in Scientific Computing // PLoS Computational Biology. 2017. Vol. 13, No. 6. e1005510.
8. Mann H. B. Nonparametric Tests Against Trend // Econometrica. 1945. Vol. 13, No. 3. P. 245–259.
9. Sen P. K. Estimates of the Regression Coefficient Based on Kendall's Tau // Journal of the American Statistical Association. 1968. Vol. 63, No. 324. P. 1379–1389.
10. Zippenfenig P. Open-Meteo.com Weather API. Zenodo, 2023. URL: <https://doi.org/10.5281/zenodo.7970649>
11. Hersbach H. et al. The ERA5 Global Reanalysis // Quarterly Journal of the Royal Meteorological Society. 2020. Vol. 146, No. 730. P. 1999–2049.
12. GitHub Actions documentation. GitHub Docs. URL: <https://docs.github.com/en/actions>
13. Pogoda i Klimat. Climate of Astana (station 35188), 1991–2020 normals. URL: <http://www.pogodaiklimat.ru/climate/35188.htm>
14. Kazhydromet. Annual Bulletin of Monitoring Status and Climate Change of Kazakhstan: 2023. Astana, 2024. 90 p. URL: https://kazhydromet.kz/uploads/files/1891/file_en/675274f0f001bezhegodnyy-byulleten-ik-za-2023_eng_ot_27-11-2024.pdf